Building a mobile-first Indic LLM: a comprehensive technical blueprint

A purpose-built 2B-parameter model with an Indic-optimized tokenizer is the proven path to deploying high-quality Hindi+English AI on mobile phones. Sarvam-1 demonstrated in late 2024 that a 2B model with a custom Indic tokenizer outperforms both Gemma-2-2B and Llama-3.2-3B on every Indic benchmark — and competes with Llama-3.1-8B at one-quarter the size. Sarvam AI sarvam At 4-bit quantization, this model class compresses to **under 1 GB**, fitting comfortably on mid-range Android devices while generating **15–30 tokens/second** on flagships. The key insight: tokenizer efficiency matters more than raw parameter count for Indian languages. Hugging Face Standard tokenizers consume 4–8 tokens per Hindi word, while Indic-optimized tokenizers achieve **1.4–2.1** arXiv — a 3–5× improvement Hugging Face that directly translates to faster inference, longer effective context, and smaller model footprint. Hugging Face +2 This report covers everything needed to build this system: existing models, tokenizer design, architecture choices, training strategy, quantization, modular language packs, deployment frameworks, benchmarks, and a build-versus-adapt decision matrix.

---

1. The Indic LLM landscape has matured rapidly

The open-source ecosystem for Indian language models exploded in 2024–2025. The table below summarizes the models most relevant to mobile deployment.

Model	Org	Params	Languages	Mobile viable?	Key strength
Sarvam-1	Sarvam AI	2B	10 Indic + En	✅ Excellent	Purpose-built Indic tokenizer (1.4–2.1 fertility); SOTA at 2B
Llama 3.2	Meta	1B / 3B	Hindi + 7 langs	✅ Good	Mobile-first design; Hindi officially supported
Navarasa 2.0	Telugu LLM Labs	2B	15 Indic + En	✅ Good	Broadest Indic coverage at 2B (Gemma fine-tune)
Nemotron-4-Mini-Hindi	NVIDIA	~2.6B active	Hindi/En/Hinglish	⚠️ Borderline	Pruned from 15B; best Hindi in 2–8B range
Qwen 2.5	Alibaba	0.5B–3B	Multilingual	✅ Good base	Apache 2.0; strong multilingual foundation
SmolLM2	Hugging Face	1.7B	English	✅ Size ideal	Apache 2.0; needs Indic training
Gemma 3n	Google	2B/4B(E)	140+ langs	✅ Good	Mobile-first; multimodal; broad language support
IndicTrans2	AI4Bharat	~200M distilled	22 Indic + En	✅ Excellent	Best Indic translation; MIT license
IndicBERT v2	AI4Bharat	278M	23 Indic + En	✅ Excellent (NLU)	Small encoder for classification/NER

Sarvam-1 stands out as the reference implementation. Trained from scratch on **4 trillion tokens** (half Indic) using 1,024 H100 GPUs, it demonstrated that a custom 68K-token vocabulary achieves **2–4× better token efficiency** than multilingual tokenizers. Hugging Face +2 On IndicGenBench, translated MMLU, ARC-Challenge, and TriviaQA, Sarvam-1 at 2B outperforms Gemma-2-2B and Llama-3.2-3B Sarvam AI while running **4–6× faster** inference than 8B-class models. AI digitalnews +2

OpenHathi (Sarvam AI, 7B) was historically important as India's first open Hindi LLM — built on Llama-2 with a 48K extended tokenizer Analytics Vidhya +3 — but is too large for mobile. **Airavata** (AI4Bharat, 7B) added instruction-tuning on top. AI digitalnews +3 **Krutrim** (Ola) released Krutrim-2 at 12B with 22-language support and strong benchmarks, but again too large. The **Hanooman/BharatGPT** consortium announced 1.5B–40B models but actual open weights remain unconfirmed.

For a mobile-first system, the viable starting points narrow to Sarvam-1, Llama 3.2 1B/3B, Navarasa 2.0, Qwen 2.5 1.5B, or training a new model from scratch following Sarvam-1's architecture.

2. Tokenizer efficiency is the single biggest lever

The most critical design decision for an Indic mobile LLM is the tokenizer. Standard LLM tokenizers were optimized for English and Latin scripts. (arXiv) When applied to Devanagari, Bengali, Telugu, or Malayalam, they produce **catastrophically high token counts** — a single Hindi sentence can consume 5–10× the tokens of its English equivalent, wasting context window, compute, and memory. (sarvam)

Fertility benchmarks (tokens per word — lower is better):

Tokenizer	English	Hindi	Odia	Bengali	Telugu	Malayalam
LLaMA-3/4	~1.3	~3–5	**10.5**	~4–6	~5–7	~5–8
GPT-4o (o200k)	~1.2	~2.5–3	~4–5	~3–4	~3–5	~4–5
Sarvam-1 (68K)	~1.4	**1.6**	**~2.0**	**~1.8**	**~2.0**	**~2.1**
IndicSuperTokenizer	~1.2	**~1.4**	**~1.6**	**~1.5**	**~1.5**	**~1.6**

IndicSuperTokenizer (arXiv:2511.03237) achieves state-of-the-art Indic fertility — **39.5% better** than LLaMA-4 and **18% better** than SUTRA (arXiv) — by combining subword and multi-word (superword) tokenization with language-specific pre-tokenization. This translates to a measured **44% improvement in inference throughput**.

Recommended tokenizer strategy

Use **SentencePiece BPE** with these specifications:

- **Vocabulary size**: 64K–96K tokens (128K if embedding sharing is used). Research shows 128K is the most balanced for Indic multilingual — best tradeoff between fertility and embedding matrix overhead. Reserve **4K–8K slots** for future language expansion.
- **Training corpus**: Balanced mixture across all 8 target languages plus English and code. Apply the iterative data mixture optimization from "The Art of Breaking Words" (arXiv:2508.06533) to equalize fertility across languages. (arXiv)
- **Pre-tokenization**: Language-aware splitting that preserves **aksara** (syllabic) boundaries in Devanagari, Bengali, Odia, Kannada, Telugu, and Malayalam. Standard byte-level BPE destroys conjunct characters — constrained BPE (MorphTok) that initializes with consonant-vowel units reduces Hindi fertility by ~0.02. (OpenReview)
- **Normalization**: Apply **IndicNLP library normalization** before tokenizer training — NFKC normalization (GitHub) plus anusvāra standardization. Normalized corpora yield measurably lower fertility (confirmed in arXiv:2506.17789).

- **Script handling for Urdu**: Nastaliq script requires Arabic/Urdu-specific normalization and right-to-left contextual shaping; SentencePiece handles this but requires dedicated processing.

Extending tokenizers for new languages

Three approaches exist for adding language support without full retraining:

1. **Vocabulary extension** (proven, used by OpenHathi): Train new SentencePiece on target language, merge vocabularies, randomly initialize new embeddings, then align through continued pretraining. (arXiv +2)
2. **TokenAdapt** (arXiv:2505.09738): Model-agnostic transplantation using hybrid heuristic initialization — decomposes new tokens through old tokenizer and averages embeddings. Cuts perplexity ratios by 2× versus baselines with zero additional training. (Hugging Face)
3. **ZeTT (Zero-Shot Tokenizer Transfer)**: Trains a hypernetwork predicting embeddings for any new tokenizer; remaining gap closed with <1B tokens of continued training. (arXiv) (Medium)

For the modular language pack architecture, the best approach is to **build a unified tokenizer covering all 8 languages from the start** (following Sarvam-1's approach), eliminating the need for tokenizer extension in language packs entirely.

---

3. Architecture should be deep, thin, and adapter-ready

The optimal transformer configuration at 1–3B

Modern research converges on a clear recipe for mobile-optimized LLMs. **MobileLLM** (Meta) established that for sub-3B models, depth matters more than width (Hugging Face) — "deep and thin" architectures outperform "wide and shallow" alternatives. (github) (InfoQ) Combined with lessons from Phi-3 (data quality over parameter count), (arXiv) TinyLlama (aggressive over-training), and SmolLM2 (multi-stage data mixing), (github +2) the recommended architecture is:

Component	Specification	Rationale
Architecture	Dense decoder-only transformer	MoE too memory-heavy for mobile
Layers	28–36	Deeper > wider at this scale (MobileLLM)
Hidden dim	2048–2560	Matches 2–3B parameter budget
Query heads	24–32	Standard for this scale
KV heads (GQA)	4–8	**Reduces KV-cache by 4–8×** vs MHA
Head dim	64	Power-of-2 for NEON SIMD alignment
FFN	SwiGLU, 3.5–5.5× hidden	Industry standard; fused kernels available
Positional encoding	RoPE	Best ecosystem support; extensible context
Normalization	RMSNorm (pre-norm)	Standard for modern transformers
Embedding sharing	Input/output tied	**Saves 20–25% of parameters** — critical at this scale
Attention pattern	3:1 local:global interleaving	MobileLLM-Pro: 512-token sliding window for local
Vocab size	64K–96K	Indic-optimized BPE (see §2)
Context	4K base, extensible to 32K+ via YaRN	Practical for mobile; saves KV-cache memory

Why not Mixture of Experts?

MoE is theoretically attractive for language-specific routing — research from Bandarkar et al. (arXiv:2510.04694) shows MoE models naturally develop **language-specific routing in early and late layers**. (arXiv) However, all expert weights must reside in memory even when only 2 of 8 are active. (github) A small MoE (8 experts × 400M active = 3.2B total) would consume the full mobile memory budget for weights alone, leaving nothing for KV-cache. **LoRA adapters achieve the same language-routing effect at 1/100th the memory cost.** (Runpod)

LoRA adapters as downloadable language modules

The architecture should support **hot-swappable LoRA adapters** for language-specific adaptation. Size estimates for a 2B model:

LoRA rank	Trainable params	Size (FP16)	Size (INT8)
8 (Q/V only)	~5.5M	~11 MB	~5.5 MB
16 (all linear)	**~11M**	**~22 MB**	**~11 MB**
32 (all linear)	~22M	~44 MB	~22 MB
64 (all linear)	~44M	~88 MB	~44 MB

Rank 16 applied to all linear layers is the recommended configuration — ~22 MB per language, achieving **90–97% of full fine-tuning quality**. Research consistently shows this is sufficient for language adaptation. Emerging AI Hubs Apple Intelligence and Google Android AICore already use this exact "one base + many LoRA adapters" pattern in production. Emerging AI Hubs

For runtime, two switching approaches exist:

- **Merged LoRA** (simpler): Pre-merge adapter weights into base model; Inferless store separate files per language (~1 GB each at Q4). Cold-swap takes ~1–2 seconds.
- **Dynamic LoRA** (more flexible): Keep base model loaded, apply adapter deltas at inference ($h = W_0x + BAx$). Datawizz **MobiLoRA** (ACL 2025) optimizes this for mobile with prefix matching and delta encoding. Hot-swap takes <100ms.

Knowledge distillation pipeline

If building from scratch isn't feasible, distillation from a larger Indic-tuned model is highly effective:

1. **White-box logit distillation** from Llama 3.1 70B (Indic-tuned) → 2–3B student using reverse KLD (MiniLLM approach — prevents student from overestimating low-probability regions) arXiv
2. **"Distilling Step-by-Step"** (Google, ACL 2023): Extract chain-of-thought rationales from teacher; a 770M T5 outperformed few-shot 540B PaLM using this approach Google Research
3. **Language-specific LoRA training** using QLoRA on the student model
4. Expected quality retention: **75–85%** at 23:1 compression (70B→3B); **85–95%** at 2–7:1 (7B→1–3B)

---

4. Training data and strategy for 8 languages

Available data per language

Combining Sangraha (AI4Bharat's 251B-token corpus, CC-BY-4.0), GitHub IndicCorp v2, CC-100, OSCAR, and Wikipedia:

Language	Available tokens	Resource level	Notes
English	1T+	Ultra-high	Effectively unlimited
Hindi	50–80B	High	Sangraha + IndicCorp well-covered
Bengali	15–25B	Medium-high	Good web corpus
Telugu	10–20B	Medium	Decent coverage
Malayalam	10–20B	Medium	Decent coverage
Kannada	10–18B	Medium	IndicCorp v1 has 713M tokens
Urdu	5–15B	Medium-low	Nastaliq script challenges
Odia	**2–5B**	**Low**	Only 107M in IndicCorp v1; 18K Wikipedia articles

Sangraha is the gold-standard training corpus. (Analytics India Magazine) Its three components — Verified (human-verified websites + OCR PDFs), Unverified (perplexity-filtered web crawl), and Synthetic (IndicTrans2-translated English Wikipedia) — cover 22 languages at varying quality levels. (GitHub) (Hugging Face) The Setu pipeline (Apache Spark-based) handles Indic-specific cleaning: Unicode canonicalization, code-mixed text detection, and OCR quality filtering. (GitHub)

Odia requires special treatment

Odia is the most data-constrained target language. (Kyoto-u) Recommended mitigations:

- **Synthetic data via IndicTrans2**: Translate English/Hindi content to Odia (Sangraha Synthetic produced ~90B tokens across 14 languages using this approach) (arXiv)
- **Cross-lingual transfer**: Bengali and Hindi share structural similarities with Odia
- **3–5× upsampling** during training (standard practice for low-resource languages)
- **Quality over quantity**: Prioritize Sangraha Verified's human-verified Odia data

Train all languages simultaneously — do not stage

A critical finding from "Revisiting Multilingual Data Mixtures in Language Model Pretraining" (arXiv:2510.25947, 2025): **curriculum learning (staging language introduction) does NOT mitigate negative interference**. English-first-then-multilingual showed no benefit over all-at-once training. (arXiv) The recommended data mix uses **temperature-based sampling** (T=0.3–0.7):

- English: 30–40%
- Hindi: 25–30%
- Bengali, Telugu, Malayalam, Kannada: 6–8% each
- Urdu: 5–6%

- Odia: 5–6% (heavily upsampled from actual data proportion)
- Code-mixed Hindi-English: 2–3%

Quality staging is valuable: start with higher-quality data (Sangraha Verified, Wikipedia) and gradually introduce noisier web-crawled data.

Continual pretraining versus from scratch

Approach	Compute cost	Timeline	Quality potential
From scratch (2B, ~2T tokens)	$50K–$100K	4–6 months total	Highest (full tokenizer control)
Continual pretraining (2–3B base, 50–100B tokens)	$6K–$14K	2–4 weeks	Good (constrained by base tokenizer)
LoRA/QLoRA fine-tuning only	$50–$1,500	Days	Moderate (limited adaptation depth)

Continual pretraining saves 5–10× over training from scratch. (arXiv) (amd) OpenHathi demonstrated the approach: extend Llama-2's tokenizer, align embeddings via translation with LoRA, then do bilingual next-token prediction. (Xcelore +4) To prevent catastrophic forgetting, include **10–20% English replay data** (arXiv) (AI Business) and consider freezing bottom layers. (Emergent Mind +2)

Alignment for Indian languages

No large-scale native Indic preference datasets exist. The practical standard:

1. **SFT** on IndicAlign-Instruct (AI4Bharat's 74.7M prompt-response pairs across 20 languages) (AI4Bharat +2)
2. **DPO** on translated preference pairs (HH-RLHF, UltraFeedback) via IndicTrans2 + IndicAlign-Toxic (arXiv)
3. Include English alignment data alongside Hindi — NVIDIA found this significantly outperforms Hindi-only SFT (arXiv)

---

5. Quantization must be handled carefully for Indic scripts

Model sizes at different quantization levels

Quant level	~Bits/weight	1.5B model	3B model
Q5_K_M	~5.5	~1.05 GB	~2.1 GB

Quant level	~Bits/weight	1.5B model	3B model
Q4_K_M	**~4.85**	**~0.95 GB**	**~1.88 GB**
Q3_K_M	~3.5	~0.72 GB	~1.4 GB
Q2_K	~2.7	~0.57 GB	~1.1 GB

The **3B Q4_K_M configuration at 1.88 GB** is confirmed to fit under the 2 GB target. A 1.5B Q4_K_M at ~0.95 GB provides the most headroom for KV-cache and runtime memory.

Indic languages are disproportionately hurt by aggressive quantization

This is a critical finding that shapes the entire quantization strategy. Three independent studies confirm it:

- **Marchisio et al. (EMNLP 2024)**: Non-Latin scripts showed **-1.9% relative drop** versus -0.7% for Latin scripts. Automatic metrics underestimate the damage by up to **10×** compared to human evaluation (1.7% automatic drop → 16.0% human-measured drop for Japanese).
- **arXiv:2508.20893**: Low-resource Indic languages lose **>10 COMET points** at 2-bit quantization. GGUF variants provide the most consistent performance even at aggressive compression.
- **arXiv:2601.18306**: Using **multilingual/Indic calibration data** during quantization improves perplexity by up to **3.52 points** compared to English-only calibration. (arXiv)

Minimum recommended quantization for acceptable Hindi quality: Q4_K_M (4-bit). Going to Q3 or Q2 risks significant degradation. If file size permits, **Q5_K_M is preferred** — the quality gap between Q4 and Q5 is more significant for Indic than for English. (arXiv)

Always use Hindi/multilingual calibration data when generating importance matrices for GGUF quantization. (arXiv) This single optimization has the largest impact on Indic language quality.

Memory budget on mobile devices

Configuration	Model size	KV-cache (2K ctx)	Runtime	Total RAM
1.5B Q4_K_M	0.95 GB	~150 MB	~300 MB	**~1.4 GB**
3B Q4_K_M	1.88 GB	~250 MB	~400 MB	**~2.5 GB**

KV-cache with GQA (4–8 KV heads) at 2K context adds ~150–250 MB. Sliding window attention caps this regardless of actual context length. (Medium) KV-cache quantization to INT8 halves memory further.

Minimum device requirements: 4 GB RAM for 1.5B Q4; 6 GB RAM for 3B Q4. **Recommended**: 6–8 GB for comfortable operation with OS overhead.

---

6. Modular language packs via LoRA adapters

The recommended architecture for downloadable language support:

```
Base App + Hindi+English Model  (~1.0 GB, 2B at Q4_K_M)
  │
  ├── Odia Language Pack      (~15-25 MB)
  │   ├── odia_lora_adapter.safetensors   (~10-20 MB, rank 16)
  │   └── odia_metadata.json              (<1 KB)
  │
  ├── Bengali Language Pack    (~15-25 MB)
  ├── Telugu Language Pack     (~15-25 MB)
  ├── Urdu Language Pack       (~15-25 MB)
  ├── Kannada Language Pack    (~15-25 MB)
  └── Malayalam Language Pack  (~15-25 MB)
```

Total system: ~1.0 GB base + 6 × ~20 MB adapters = **~1.12 GB** for all 8 languages — well within mobile storage constraints.

If the base tokenizer already covers all target languages (recommended), no tokenizer extension is needed in language packs — just the LoRA weights. The user flow: select language in settings → download 15–25 MB pack from CDN → adapter loaded into memory → generation uses adapted weights. Once downloaded, packs work fully offline.

Existing production systems validate this approach: **Apple Intelligence** uses LoRA adapters for on-device language/task adaptation, Emerging AI Hubs **Google Android AICore** supports adapter-based specialization, and **MobiLoRA** (ACL 2025) specifically optimizes multi-LoRA inference for mobile with prefix matching and utility-based KV-cache eviction.

For **X-Mod-style routing** (Meta/FAIR), language-specific modules are pre-trained from the start — enabling post-hoc language addition with no performance drop. OpenReview GitHub The `model.set_default_language("hi_XX")` API pattern is clean and proven across 81 languages. GitHub

---

7. llama.cpp plus WebLLM is the optimal deployment stack

Framework comparison for React Native + Web

Feature	llama.cpp (via llama.rn)	ExecuTorch	WebLLM	wllama
Platform	Android + iOS	Android + iOS	Browser	Browser
RN binding	**llama.rn** (801 stars, v0.11.2)	**react-native-executorch** (Software Mansion)	N/A	N/A

Feature	llama.cpp (via llama.rn)	ExecuTorch	WebLLM	wllama
Model format	GGUF (any HF model)	.pte (PyTorch export)	MLC compiled	GGUF
GPU accel	Metal (iOS) + OpenCL (Android)	12+ backends (CoreML, QNN, ARM)	WebGPU	None (WASM only)
Maturity	★★★★★	★★★★☆ (1.0 GA Oct 2025)	★★★★☆	★★★☆☆
Model ecosystem	Largest (any GGUF)	Growing	Large	Any GGUF

Performance benchmarks (tokens/second, decode)

Device	1.5B Q4	3B Q4
Apple A18 (iPhone 16)	~25–35 t/s	~15–22 t/s
Apple A17 Pro (iPhone 15 Pro)	~20–30 t/s	~12–18 t/s
Snapdragon 8 Gen 3	~15–22 t/s	~8–14 t/s
Snapdragon 8 Gen 4	~18–25 t/s	~12–18 t/s
Mid-range (SD 750G)	~4–8 t/s	~2–4 t/s
Browser (WebLLM, laptop GPU)	~20–40 t/s	~10–20 t/s
Browser (wllama WASM)	~3–8 t/s	~2–5 t/s

Recommended architecture for the React Native app

```
React Native App
├── Mobile (Android/iOS)
│   ├── Primary: llama.rn (GGUF models)
│   │   └── Vercel AI SDK wrapper: @react-native-ai/llama
│   └── Alternative: react-native-executorch (.pte)
├── Web
│   ├── Primary: WebLLM (WebGPU) — 80% native speed
│   └── Fallback: wllama (WASM) — broader browser support
└── Unified API Layer
    └── @react-native-ai (Callstack) — Vercel AI SDK compatible
```

llama.rn (mybigday/llama.rn) is the primary recommendation: Medium most mature RN binding, works with any GGUF model, supports Metal (iOS) and OpenCL (Android Adreno 700+), Expo plugin support, pre-built binaries. GitHub **Callstack's @react-native-ai/llama** wraps it with Vercel AI SDK compatibility and model download management. npm

For web deployment, **WebLLM** retains **up to 80% of native performance** via WebGPU. arXiv Fallback to **wllama** (WASM, CPU-only) for browsers without WebGPU support npm (notably iOS Safari as of March 2026).

---

8. Benchmarking Indic quality requires purpose-built evaluations

Use this evaluation suite:

- **IndicGenBench** (ACL 2024): The most comprehensive current benchmark — 29 languages, 13 scripts, covering cross-lingual summarization, translation, reading comprehension, and QA ACL Anthology
- **IndicMMLU-Pro** (Jan 2025): Multi-task understanding across 9 Indic languages, extending MMLU-Pro framework arXiv
- **Flores-200**: Standard for translation quality (chrF++ and BLEU)
- **IndicGLUE**: 11-language NLU benchmark (genre classification, headline prediction, NER, paraphrase detection, Winograd NLI) ACL Anthology
- **MILU**: Comprehensive evaluation of 40+ models; GPT-4 achieved highest at 72% accuracy Analytics India Magazine arXiv
- **GLUECoS**: Hindi-English code-mixed benchmark (critical for real Indian digital communication) arXiv

Quality achievable by parameter count for Hindi+English:

Parameters	Expected quality	Evidence
0.5–1B	Basic instruction following; limited Hindi depth	Qwen3-0.6B shows capability but weak Indic
1.5–2B	**Strong Hindi+English**	Sarvam-1 (2B) matches 8B generalist models on Indic tasks
3B	Excellent for complex tasks	Llama-3.2-3B + Indic fine-tuning
7–8B	Near-frontier quality but too large for mobile	Llama-3.1-8B

The critical finding: **a purpose-built 2B model outperforms generic 3B models on Indic tasks**, proving architecture + tokenizer + data quality > raw parameter count. Sarvam AI

---

9. The build-versus-adapt decision tree

The recommended path: hybrid approach

Take an existing architecture, build a custom Indic tokenizer, and continual-pretrain from scratch. This is exactly what Sarvam-1 did, and it is the highest-confidence approach.

Decision factor	Build from scratch	Adapt existing	Hybrid (recommended)
Tokenizer control	✅ Full	❌ Constrained	✅ Full
Compute cost	$50K–$100K	$6K–$14K	$20K–$50K
Timeline	4–6 months	2–4 weeks	2–3 months
Hindi quality	Highest potential	Good (limited by tokenizer)	High
Engineering team	5–10 ML engineers	2–3 engineers	3–5 engineers
Risk	Higher (training instability)	Lower	Moderate

If budget is very limited (<$15K): Take **Qwen 2.5 1.5B** (Apache 2.0, best multilingual tokenizer among open models) and continual-pretrain on 50–100B Indic tokens from Sangraha. Accept suboptimal tokenizer efficiency as a tradeoff. This gets a working model in weeks for under $10K in compute.

If budget allows ($20K–$100K): Follow the Sarvam-1 playbook — Llama-style architecture with custom 64K–96K Indic tokenizer, trained on 200B–2T tokens. Use **IndiaAI Mission compute** (subsidized to <$1/hour per GPU) if eligible. Equentis

Best base model for adaptation (if not building tokenizer from scratch):

1. **Qwen 2.5 1.5B**: Best multilingual tokenizer foundation, Apache 2.0, strong baseline
2. **Llama 3.2 3B**: Most tunable (biggest gains from fine-tuning per Distil Labs benchmarks), Distillabs native Hindi support
3. **Llama 3.2 1B**: Smallest viable generative model, most mobile-friendly
4. **SmolLM2 1.7B**: Best license (Apache 2.0), well-documented architecture, but English-only

---

10. Government infrastructure and the competitive landscape

India's AI compute infrastructure has reached critical mass. The IndiaAI Mission allocated **₹10,371**

crore (~$1.25 billion) over 5 years and procured **14,517 GPUs** in its first round from Reliance Jio, Yotta, E2E, NxtGen, and Tata Communications — with GPU costs subsidized to under $1/hour (cheapest globally). Sarvam AI was selected as the first company, receiving **4,000 GPUs for 6 months** plus ₹246.72 crore in funding.

Bhashini, the National Language Translation Mission, processes **300 million+ monthly translations** across 22 scheduled languages. AI4Bharat runs Bhashini's Data Management Unit, collecting 15,000 hours of transcribed speech from 400+ districts and creating 2.2 million translation pairs.

Key players to monitor:

- **Sarvam AI**: Released Sarvam-1 (2B), Sarvam-M (24B), and in Feb 2026 launched 30B and 105B MoE models. Their "Edge Intelligence" product line explicitly targets on-device deployment.
- **AI4Bharat (IIT Madras)**: India's premier open-source lab — IndicTrans2/3, Sangraha (251B tokens), IndicVoices, BhasaAnuvaad (44,400 hours speech), and new 270M/1B/4B encoder models.
- **Krutrim (Ola)**: India's first AI unicorn; Krutrim-2 (12B) open-sourced with 22-language support; building India's largest supercomputer.
- **NVIDIA**: Nemotron-4-Mini-Hindi-4B (open license) deployed by Tech Mahindra, IITs, and healthcare companies.
- **Google**: Gemma 3n (2B/4B effective) with 140+ language support is the strongest mobile-first multimodal option from a major lab.

Conclusion: an actionable implementation roadmap

The research conclusively supports a **2B-parameter model with a custom Indic tokenizer** as the optimal configuration — validated by Sarvam-1's state-of-the-art results. Here is the prioritized implementation plan:

Phase 1 (Weeks 1–4): Tokenizer and data pipeline. Build a 64K–96K SentencePiece BPE tokenizer on balanced Indic+English data from Sangraha, targeting ≤2.0 fertility across all 8 languages. Set up the Setu cleaning pipeline for data processing.

Phase 2 (Weeks 4–10): Model training. Use a Llama-style architecture (28–36 layers, GQA, SwiGLU, embedding sharing) and train on 200B+ tokens. If adapting an existing model, start from Qwen 2.5 1.5B with extended tokenizer and continual-pretrain on 50–100B tokens. **Estimated compute: $10K–$50K** depending on approach.

Phase 3 (Weeks 10–12): Alignment and adapters. SFT on IndicAlign-Instruct, DPO on translated preference data. Train rank-16 LoRA adapters for each optional language (~20 MB each).

Phase 4 (Weeks 12–14): Quantization and deployment. Quantize to GGUF Q4_K_M using Hindi+multilingual calibration data. Deploy via llama.rn (React Native), WebLLM (browser), with wllama WASM fallback. Target: ~0.95 GB (1.5B) or ~1.88 GB (3B) base model + ~20 MB per language pack.

The non-negotiable insight: tokenizer efficiency determines everything downstream — inference speed, context length, model size, and ultimately user experience. A 2B model with 1.5× Hindi fertility will outperform a 3B model with 5× fertility on every metric that matters for mobile deployment. Build the tokenizer first, and build it right.